

Machine learning final project

111705010 汪俞宏

Outline

1. Introduction
2. Dataset Overview & Pre-processing
3. Model Design
4. Training Setup
5. Experimental Results
6. Challenges and Solutions
7. Conclusion & Future Work

Introduction

- The challenge is to **detect and localize surface defects** on metallic industrial products using deep learning.
- Build an accurate **object detection model** that can Identify bounding boxes around defects and

Dataset Overview & Pre-processing

The dataset was split as follows:

- **Training Set:** All 1,440 images used for training.
- **Validation:** Optionally split 10–20% manually from training for local evaluation.
- **Test Set:** 360 unlabeled images used only for inference and submission.

Dataset Overview & Pre-processing

T.ToImage()

Converts a PIL image into a format (`torchvision.tv_tensors.Image`) that can be used with `transforms.v2`.

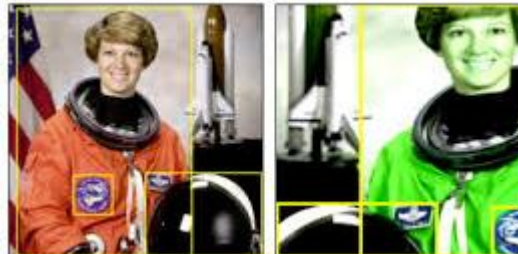
T.RandomHorizontalFlip(p=0.5)

With 50% probability, it flips the image and its bounding boxes horizontally. Helps model learn left-right

T.RandomPhotometricDistort()

Applies random changes to brightness, contrast, saturation, and hue.

Simulates lighting or camera color variation, which boosts robustness to real-world image capture conditions.



Model Design

Architecture Used

We used the Faster R-CNN object detection model with a ResNet-50 backbone and FPN (Feature Pyramid Network) as provided by the `torchvision.models.detection` module

Faster R-CNN is a two-stage detector:

Region Proposal Network (RPN) proposes candidate object regions.

Second-stage classifier + regressor refines the predictions.

Backbone: ResNet-50 (He et al., 2016) pre-trained on COCO dataset.

Feature Pyramid Network (FPN): Enhances the model's ability to detect small and large objects by combining low- and high-resolution feature maps.

Model Design

Modifications to the Base Model

Component	Improvement	Context/Explanation
<code>NEUDETDataset</code> class	Data Augmentation Added	Introduced realistic transformations like flipping, jitter, and cropping to generalize better.
<code>train_model</code> function	Gradient Clipping	Prevents exploding gradients by limiting the norm, stabilizing training.
	Learning Rate Scheduler	Uses <code>ReduceLROnPlateau</code> to reduce learning rate on plateau in loss, improving convergence.
	Best Model Saving	Automatically saves the best-performing model to avoid overfitting and preserve progress.
	AdamW Optimizer	Switch from plain Adam to AdamW for better regularization with weight decay.

Model Design

Modifications to the Base Model

Component	Improvement	Context/Explanation
<code>predict_and_export</code>	Confidence Threshold	Filters predictions based on score threshold (e.g., 0.5) to reduce false positives.
	More Robust Prediction Output Handling	Handles images with no detections to avoid crashing and ensures clean CSV output.
	Improved Visualization	Bounding boxes drawn with scores and labels; output image saved for review.
<code>main</code> script	Checkpoint Handling	Loads existing model weights if available to resume training instead of starting over.
Code Structure	Cleaner Modular Design	Separated augmentation, loading, training, and prediction into reusable components.

Training Setup

Loss Function: The model uses the built-in multi-task loss from `torchvision.models.detection`, which is composed of :

- Classification loss (cross-entropy)
- Bounding box regression loss (smooth L1 loss)
- RPN objectness loss
- RPN bounding box regression loss

Optimizer: AdamW optimizer is used:

```
input :  $\gamma$ (lr),  $\beta_1, \beta_2$ (betas),  $\theta_0$ (params),  $f(\theta)$ (objective),  $\epsilon$  (epsilon)  
         $\lambda$ (weight decay), amsgrad, maximize  
initialize :  $m_0 \leftarrow 0$  (first moment),  $v_0 \leftarrow 0$  ( second moment),  $v_0^{max} \leftarrow 0$   
  
for  $t = 1$  to ... do  
    if maximize :  
         $g_t \leftarrow -\nabla_{\theta} f_t(\theta_{t-1})$   
    else  
         $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$   
     $\theta_t \leftarrow \theta_{t-1} - \gamma \lambda \theta_{t-1}$   
     $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$   
     $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$   
     $\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$   
    if amsgrad  
         $v_t^{max} \leftarrow \max(v_{t-1}^{max}, v_t)$   
         $\widehat{v}_t \leftarrow v_t^{max} / (1 - \beta_2^t)$   
    else  
         $\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$   
     $\theta_t \leftarrow \theta_t - \gamma \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$   
  
return  $\theta_t$ 
```

Training Setup

Learning Rate Scheduler: ReduceLROnPlateau is used to reduce LR when validation loss plateaus

Training Parameters

Epochs: 100

Batch Size: 4

Gradient Clipping: Used to prevent exploding gradients: Tools Used

Framework: PyTorch

Experimental Results , Challenges and Solutions

Private Score : Achieved a private leaderboard score of 0.5882 on the Kaggle test set.

This result was obtained without any custom modifications, using the default pretrained Faster R-CNN model and a fixed confidence threshold of 0.5.

Observed Issues 1 : A significant number of false negatives and incorrect detections were observed in images containing crazing defects.

Possible reasons:

- Crazing often has fine-grained patterns that resemble background noise.
- The model may not have learned the distinguishing features well due to data imbalance or visual similarity to other defects or background.

```
[RESULT] AP per class:  
- crazing: 0.0000  
- inclusion: 0.9981  
- patches: 0.9968  
- pitted_surface: 0.9971  
- rolled-in_scale: 0.9980  
- scratches: 0.9973  
  
[RESULT] mAP (mean AP over all classes): 0.8312
```

Experimental Results , Challenges and Solutions

Observed Issues 2 Inconsistent Confidence Score Distribution Across Images

During evaluation, a notable variation in confidence scores was observed across different images in the test set:

- In some images, the model outputs high-confidence predictions that accurately localize defects
- In other images, the model generates reasonably correct bounding boxes around defects, but these boxes are assigned low Confidence score
- This highlights a limitation of applying a single, global confidence threshold uniformly across the dataset
- It tends to keep only the most confident detections, but at the cost of missing valid predictions with lower confidence scores

Experimental Results , Challenges and Solutions

Solution for Observed Issues 2 : tune a model just for crazing dataset and make some improvement

Key Fixes Implemented

1. Better Data Augmentation : `T.RandomHorizontalFlip`, `T.RandomVerticalFlip`, `T.ColorJitter`, `T.RandomRotation`, `T.RandomAdjustSharpness`, `T.RandomAutocontrast`
2. Image Normalization
3. Lower Confidence Threshold
4. Gradient Clipping : Prevents exploding gradients, ensuring stable learning and preventing weight updates that overshoot, which can cause underfitting.
5. Model Checkpointing on Best Loss
6. Visual Debugging Support

Experimental Results , Challenges and Solutions

Solution for Observed Issues 1 eliminate the global threshold and train a data-driven threshold predictor.

1. Extract Confidence and Match Quality Per Threshold

- For each prediction image : Apply 19 thresholds (from 0.05 to 0.95).
- Compute how well these predictions match the ground truth using $\text{IoU} > 0.5$.
- Compute F1 score at each threshold.

2. Train Threshold Prediction Model

- Uses `df_all` which contains features at all thresholds and their F1 scores.
- Predicts the threshold value ($y = \text{df_all}[\text{'threshold'}]$) based on features:

Experimental Results , Challenges and Solutions

Solution for Observed Issues 1 eliminate the global threshold and train a data-driven threshold predictor.

2. Train Threshold Prediction Model

- Uses Gradient Boosting Regressor from sklearn:
- This model learns a mapping from these features to the best threshold to use.
- Gradient Boosting is a tree-based ensemble model, good at capturing nonlinear relations.

3. Apply the Learned Per-Image Thresholds

- For each image's predictions: Compute the features (num_preds, avg_score, max_score, min_score).
- Predict the best threshold for this image using the trained model.
- Filter out predictions below the predicted threshold.

Experimental Results , Challenges and Solutions

Solution for Observed Issues 1 eliminate the global threshold and train a data-driven threshold predictor.

Result of the final predict threshold :

```
thresholds_dict = {  
    'crazing': 0.5,  
    'inclusion': 0.83,  
    'patches': 0.63,  
    'pitted_surface': 0.6,  
    'rolled-in_scale': 0.75,  
    'scratches': 0.8,  
}
```

The final public map score improved to **0.7583**, demonstrating significant enhancement over the improvement of the two Observed issues

Conclusion & Future Work

Conclusion :

- In this project, we developed a deep learning-based object detection model to detect and localize surface defects on metallic industrial products. Leveraging the Faster R-CNN architecture with a ResNet-50 backbone and FPN
- We implemented targeted solutions including enhanced data augmentation, gradient clipping, model checkpointing, and a novel data-driven threshold predictor using gradient boosting regression.
- These improvements significantly boosted the model's robustness and precision, raising the public map score to 0.7583.
- The project demonstrated the effectiveness of combining state-of-the-art detection architectures with customized training strategies and threshold optimization to tackle real-world industrial defect detection tasks.

Conclusion & Future Work

Future Work :

- Advanced Model Architectures
- Multi-Task Learning
- Semi-Supervised Learning
- Real-Time Inference Optimization
- Robustness to Varying Conditions
- Interactive Demo